Lab 5: Compiling C Programs for SimpleScalar PISA

Computer Architecture Laboratory

1 Objective

This lab shows how to write C programs, compile them for the SimpleScalar PISA little-endian target, and run them with the simulators. It follows the idea of the UMass SimpleScalar bonus compilation lab, with commands updated for this repository:

```
http://www.ecs.umass.edu/ece/koren/architecture/Simplescalar/bonuslab1.htm
```

The current repository uses a local installer script instead of the older manual compiler installation process.

2 Part 1: Install the Local PISA Compiler

Run from the cloned repository root:

```
cd /path/to/SimpleScalar
bash scripts/install_pisa_cross_compiler.sh --install-deps
```

If the required Ubuntu/Debian build packages are already installed:

```
bash scripts/install_pisa_cross_compiler.sh
```

The compiler is installed locally:

```
toolchain/sslittle-na-sstrix/bin/gcc
```

Do not use a generic `mips-linux-gnu-gcc`; it normally produces incompatible Linux ELF binaries instead of SimpleScalar PISA ECOFF/SSTrix binaries.

3 Provided Source Programs

All C programs used in this lab are included in the repository. Students should compile these files directly instead of copying program text from the PDF:

```
labs/hello_loop.c
labs/test1.c
labs/test2.c
labs/o2_o3_diff.c
```

To inspect a program before compiling it:

```
less labs/hello_loop.c
```

4 Part 2: Hello Program With a Long Loop

Use the provided source file `labs/hello_loop.c`. This program runs a long empty loop and then prints a message. Compile without optimization:

```
toolchain/sslittle-na-sstrix/bin/gcc -static -o labs/hello_noopt.pisa labs/hello_loop.c
```

Compile with `-O2`:

```
toolchain/sslittle-na-sstrix/bin/gcc -O2 -static -o labs/hello_O2.pisa labs/hello_loop.c
```

Compile with `-O3`:

```
toolchain/sslittle-na-sstrix/bin/gcc -O3 -static -o labs/hello_O3.pisa labs/hello_loop.c
```

Confirm the binary format:

```
file labs/hello_noopt.pisa labs/hello_O2.pisa labs/hello_O3.pisa
```

Expected format:

```
MIPSEL ECOFF executable
```

Run each program with `sim-safe`:

```
./sim-safe -redir:sim labs/hello_noopt_sim.out -redir:prog labs/hello_noopt_prog.out labs
    /hello_noopt.pisa
./sim-safe -redir:sim labs/hello_O2_sim.out -redir:prog labs/hello_O2_prog.out labs/
    hello_O2.pisa
./sim-safe -redir:sim labs/hello_O3_sim.out -redir:prog labs/hello_O3_prog.out labs/
    hello_O3.pisa
```

Extract instruction count and elapsed simulator time:

```
grep -E "sim_num_insn|sim_elapsed_time" labs/hello_*_sim.out
```

Fill Table 1. Questions:

Table 1: Optimization results for `hello_loop.c`

Hello program	Total instructions	Total elapsed time
No optimization		
`-O2`		
`-O3`		

1. Did you observe a difference between no optimization and `-O2`?
2. Did you observe a difference between no optimization and `-O3`?
3. Did you observe a difference between `-O2` and `-O3`?
4. Briefly describe what `-O2` and `-O3` do.
5. Plot a histogram comparing the optimization levels.

5 Part 3: Integer Arithmetic Benchmark

Use the provided source file `labs/test1.c`. This program repeats a simple integer arithmetic expression inside a loop. Compile:

```
toolchain/sslittle-na-sstrix/bin/gcc -static -o labs/test1_noopt.pisa labs/test1.c
toolchain/sslittle-na-sstrix/bin/gcc -O2 -static -o labs/test1_O2.pisa labs/test1.c
toolchain/sslittle-na-sstrix/bin/gcc -O3 -static -o labs/test1_O3.pisa labs/test1.c
```

Run with `sim-safe`:

```
./sim-safe -redir:sim labs/test1_noopt_safe.out -redir:prog labs/test1_noopt_prog.out
    labs/test1_noopt.pisa
./sim-safe -redir:sim labs/test1_O2_safe.out -redir:prog labs/test1_O2_prog.out labs/
    test1_O2.pisa
./sim-safe -redir:sim labs/test1_O3_safe.out -redir:prog labs/test1_O3_prog.out labs/
    test1_O3.pisa
```

Run with `sim-profile`:

```
./sim-profile -iclass -redir:sim labs/test1_noopt_profile.out labs/test1_noopt.pisa
./sim-profile -iclass -redir:sim labs/test1_O2_profile.out labs/test1_O2.pisa
./sim-profile -iclass -redir:sim labs/test1_O3_profile.out labs/test1_O3.pisa
```

Fill Table 2 using `sim_num_insn`, `sim_elapsed_time`, and the instruction class profile.

Table 2: Optimization results for the integer arithmetic benchmark

Test1	Total instructions	Total elapsed time	Integer operations %	Floating operations %
No optimization				
`-O2`				
`-O3`				

6 Part 4: Floating-Point Arithmetic Benchmark

Use the provided source file `labs/test2.c`. This program is similar to `test1.c`, but the arithmetic variables are floating-point values. Compile:

```
toolchain/sslittle-na-sstrix/bin/gcc -static -o labs/test2_noopt.pisa labs/test2.c
toolchain/sslittle-na-sstrix/bin/gcc -O2 -static -o labs/test2_O2.pisa labs/test2.c
toolchain/sslittle-na-sstrix/bin/gcc -O3 -static -o labs/test2_O3.pisa labs/test2.c
```

Run with `sim-safe`:

```
./sim-safe -redir:sim labs/test2_noopt_safe.out -redir:prog labs/test2_noopt_prog.out
    labs/test2_noopt.pisa
./sim-safe -redir:sim labs/test2_O2_safe.out -redir:prog labs/test2_O2_prog.out labs/
    test2_O2.pisa
./sim-safe -redir:sim labs/test2_O3_safe.out -redir:prog labs/test2_O3_prog.out labs/
    test2_O3.pisa
```

Run with `sim-profile`:

```
./sim-profile -iclass -redir:sim labs/test2_noopt_profile.out labs/test2_noopt.pisa
./sim-profile -iclass -redir:sim labs/test2_O2_profile.out labs/test2_O2.pisa
./sim-profile -iclass -redir:sim labs/test2_O3_profile.out labs/test2_O3.pisa
```

Fill Table 3. Questions:

Table 3: Optimization results for the floating-point arithmetic benchmark

Test2	Total instructions	Total elapsed time	Floating operations %	Integer operations %
No optimization				
`-O2`				
`-O3`				

1. Does the floating-point benchmark show a different instruction mix from the integer benchmark?
2. Which optimization level gives the lowest instruction count?
3. Which benchmark is more useful for measuring integer operations per second?
4. Which benchmark is more useful for measuring floating-point operations per second?

7 Part 5: Program That Shows a Difference Between `-O2` and `-O3`

The earlier examples may produce the same result for `-O2` and `-O3`. This can happen when `-O2` has already performed the useful optimizations for a simple loop. This repository includes a small program that is better for observing a difference:

```
labs/o2_o3_diff.c
```

Use the provided source file `labs/o2_o3_diff.c`. The program repeatedly calls a small function inside a loop. With `-O3`, the compiler can be more aggressive about inlining the function body into the loop, reducing function-call and branch overhead. Compile:

```
toolchain/sslittle-na-sstrix/bin/gcc -O2 -static -o labs/o2_o3_diff_O2.pisa labs/
    o2_o3_diff.c
toolchain/sslittle-na-sstrix/bin/gcc -O3 -static -o labs/o2_o3_diff_O3.pisa labs/
    o2_o3_diff.c
```

Confirm the binary format:

```
file labs/o2_o3_diff_O2.pisa labs/o2_o3_diff_O3.pisa
```

Run with `sim-safe`:

```
./sim-safe -redir:sim labs/o2_o3_diff_O2_safe.out -redir:prog labs/o2_o3_diff_O2_prog.out
     labs/o2_o3_diff_O2.pisa
./sim-safe -redir:sim labs/o2_o3_diff_O3_safe.out -redir:prog labs/o2_o3_diff_O3_prog.out
     labs/o2_o3_diff_O3.pisa
```

Check that both versions produce the same program output:

```
cat labs/o2_o3_diff_O2_prog.out
cat labs/o2_o3_diff_O3_prog.out
```

Extract instruction count and elapsed simulator time:

```
grep -E "sim_num_insn|sim_elapsed_time" labs/o2_o3_diff_O2_safe.out
grep -E "sim_num_insn|sim_elapsed_time" labs/o2_o3_diff_O3_safe.out
```

Run instruction profiling:

```
./sim-profile -iclass -redir:sim labs/o2_o3_diff_O2_profile.out labs/o2_o3_diff_O2.pisa
./sim-profile -iclass -redir:sim labs/o2_o3_diff_O3_profile.out labs/o2_o3_diff_O3.pisa
```

Check instruction class percentages:

```
grep -E "load|store|uncond branch|cond branch|int computation|fp computation" labs/
    o2_o3_diff_O2_profile.out
grep -E "load|store|uncond branch|cond branch|int computation|fp computation" labs/
    o2_o3_diff_O3_profile.out
```

Fill Table 4. Questions:

Table 4: Comparison of `-O2` and `-O3` on `o2_o3_diff.c`

Version	Program output	Total instructions	Total elapsed time	Unconditional branch count
`-O2`				
`-O3`				

1. Which optimization level executes fewer instructions?
2. Which optimization level has fewer unconditional branches?

3. Why does reducing function-call overhead reduce the dynamic instruction count?
4. Why must the program output be checked before comparing performance?